DATABASE MANAGEMENT SYSTEMS

5

RELATIONAL ALGEBRA

Selection • Projection • Union • Intersection • Difference • Cartesian Product • Simple Join

A Complete, Exam-Oriented Study Guide
with Solved Examples, Diagrams & Practice Queries

Table of Contents

5. Relational Algebra — Introduction

- **5.1** Selection (σ)
- **5.2** Projection (π)
- **5.3** Union (∪)
- **5.4** Intersection (∩)
- **5.5** Difference (−)
- **5.6** Cartesian Product (×)
- **5.7** Simple Join (⋈)

5.8 Queries to be Solved — Practice Set

5.9 Quick Revision Tables

5.10 Final Exam Revision Notes

5 Relational Algebra — Introduction

Relational Algebra is a **procedural query language** used to retrieve and manipulate data stored in relational databases. It is called *procedural* because a query is expressed as a sequence of operations — the user specifies **what data is needed and how to obtain it** by combining basic operations step by step.

Every relational database is made up of **relations**. A relation can be pictured as a table, where:

- A **relation** corresponds to a table (e.g. the STUDENT relation).
- A **tuple** corresponds to a single row of the table (one record).
- An **attribute** corresponds to a column of the table (one field/property).

The most important property of relational algebra is **closure**. Closure means every operation takes one or more relations as input, and its output is **always another relation** — never just a single value or a list. Because the result of one operation is itself a relation, that result can be fed straight into the next operation as its input. This is what lets us combine simple operations, one after another, to answer more complex queries.

● REMEMBER

Relational Algebra operations always follow the pattern: **Relation(s) IN → Operation → Relation OUT.** This closure property is what makes it possible to write multi-step, nested queries.

Notation Used in this Chapter

Symbol	Operation	Meaning
σ	Selection	Selects rows (tuples) satisfying a condition
π	Projection	Selects columns (attributes)
∪	Union	Combines tuples of two relations
∩	Intersection	Common tuples of two relations
−	Difference	Tuples in one relation but not the other
×	Cartesian Product	Pairs every tuple of R with every tuple of S
⋈	Join	Combines related tuples using a join condition
R, S, ...	Relation names	Written in capital letters
A, B, ...	Attribute names	Written as they appear in the relation schema

Relational algebra operations are broadly of two kinds used in this chapter: **unary operations** (Selection, Projection) that act on a single relation, and **binary operations** (Union, Intersection, Difference, Cartesian Product, Join) that act on a pair of relations.

5.1 Selection (σ)

Meaning and Purpose

Selection extracts a subset of the **tuples (rows)** of a relation that satisfy a given condition, called the **selection condition** or **predicate**. It does not affect the number of columns — every attribute of the original relation is retained; only unwanted rows are filtered out.

General Syntax

$$\sigma_{<condition>}(R)$$

where **R** is the relation and **<condition>** is a Boolean expression made up of attribute names, constants, comparison operators (**=, ≠, <, ≤, >, ≥**), and logical operators (**AND (∧), OR (∨), NOT (¬)**).

EXAM POINT

The **output relation** of Selection has the **same schema (attributes)** as the input relation — only the number of rows may reduce.

Sample Relation Used in this Section

STUDENT(Student_ID, Name, Department, Marks)

Relation STUDENT

Student_ID	Name	Department	Marks
S1	Aarav	CSE	85
S2	Diya	ECE	72
S3	Kabir	CSE	91
S4	Meera	ME	68
S5	Rohan	ECE	78
S6	Sara	CSE	55

Example 1 — Single Condition

Requirement: Find all students whose Marks are greater than 75.

$$\sigma_{Marks>75}(STUDENT)$$

STEP 1. Scan every tuple of STUDENT.

STEP 2. Keep only tuples where the value of Marks > 75.

STEP 3. Return the matching tuples, with all four attributes intact.

σ condition (row filter)

Student_ID	Name	Department	Marks	
S1	Aarav	CSE	85	✓ kept
S2	Diya	ECE	72	✗ dropped
S3	Kabir	CSE	91	✓ kept
S4	Meera	ME	68	✗ dropped
S5	Rohan	ECE	78	✓ kept
S6	Sara	CSE	55	✗ dropped

Fig 5.1(a): Selection filters rows — green rows satisfy Marks > 75, red rows are dropped.

Result

Student_ID	Name	Department	Marks
S1	Aarav	CSE	85
S3	Kabir	CSE	91
S5	Rohan	ECE	78

Example 2 — Selecting by a Department

Requirement: Find all students belonging to the CSE department.

$$\sigma_{\text{Department='CSE'}}(\text{STUDENT})$$

Result

Student_ID	Name	Department	Marks
S1	Aarav	CSE	85
S3	Kabir	CSE	91
S6	Sara	CSE	55

Example 3 — Multiple Conditions (AND)

Requirement: Find students who belong to the CSE department **and** have Marks greater than 80.

$$\sigma_{(\text{Department='CSE'}) \wedge (\text{Marks}>80)}(\text{STUDENT})$$

STEP 1. Apply Department = 'CSE' — keeps S1, S3, S6.

STEP 2. From these, apply Marks > 80 — keeps only S1 and S3.

STEP 3. Both conditions must be true simultaneously (logical AND).

Result

Student_ID	Name	Department	Marks

S1	Aarav	CSE	85
S3	Kabir	CSE	91

Example 4 — Multiple Conditions (OR)

Requirement: Find students who are from the ME department **or** have Marks less than 60.

$$\sigma_{(Department='ME') \vee (Marks<60)}(STUDENT)$$

Result

Student_ID	Name	Department	Marks
S4	Meera	ME	68
S6	Sara	CSE	55

COMMON MISTAKE

Do not confuse Selection with Projection. Selection (σ) filters **rows** based on a condition; it never removes or changes columns.

5.2 Projection (π)

Meaning and Purpose

Projection extracts a subset of the **columns (attributes)** of a relation, discarding the rest. Unlike Selection, Projection works on the *vertical* structure of the table.

General Syntax

$$\pi_{<attribute_list>}(R)$$

where **<attribute_list>** is a comma-separated list of the attributes of R to be retained in the output.

IMPORTANT

Because relations are **sets** of tuples, if Projection produces duplicate rows (two tuples become identical once unwanted columns are removed), the **duplicates are automatically eliminated** from the result — explained in Example 3.

Selection vs Projection — Visual Difference



Fig 5.2(a): Projection keeps chosen columns (blue) and drops the rest (grey) — compare with Fig 5.1(a), which kept rows, not columns.

Selection vs Projection — Quick Comparison

Acts on	Rows (tuples)	Columns (attributes)
Symbol	σ	π
Effect on schema	No change in attributes	Fewer attributes retained
Effect on tuples	Fewer/equal tuples	Duplicates removed
Typical use	“WHICH rows qualify?”	“WHICH columns are needed?”

Example 1 — Projecting a Single Attribute

Requirement: List the names of all students (no duplicates).

$$\pi_{Name}(STUDENT)$$

Result

Name

Aarav
Diya
Kabir
Meera
Rohan
Sara

Example 2 — Projecting Multiple Attributes

Requirement: List the Name and Department of every student.

$$\pi_{\text{Name, Department}}(\text{STUDENT})$$

Result

Name	Department
Aarav	CSE
Diya	ECE
Kabir	CSE
Meera	ME
Rohan	ECE
Sara	CSE

Example 3 — Duplicate Elimination

Requirement: List only the Department values present in STUDENT.

$$\pi_{\text{Department}}(\text{STUDENT})$$

STEP 1. Read the Department value of every tuple: CSE, ECE, CSE, ME, ECE, CSE.

STEP 2. Since a relation cannot contain duplicate tuples, repeated values are merged into one occurrence.

Result — Duplicates Removed

Department
CSE
ECE
ME

Example 4 — Projection Combined with Selection

Requirement: List the names of students who scored more than 75 marks.

$$\pi_{Name}(\sigma_{Marks>75}(STUDENT))$$

STEP 1. Inner operation: σ Marks>75 (STUDENT) — selects the qualifying rows (S1, S3, S5).

STEP 2. Outer operation: π Name(...) — keeps only the Name column of that intermediate result.

Final Result

Name
Aarav
Kabir
Rohan

EXAM POINT

σ (Selection) works on **rows** and never changes attributes.
π (Projection) works on **columns** and may reduce the number of rows only through duplicate elimination.

5.3 Union (∪)

Meaning

The **Union** of two relations R and S, written **R ∪ S**, returns a relation containing all tuples that appear in R, or in S, or in both — with **duplicate tuples eliminated**, since the result must itself be a valid relation (a set of tuples).

General Syntax

```
R ∪ S
```

Union Compatibility (Precondition)

Two relations R and S can be combined using Union, Intersection or Difference only if they are **union-compatible**, which requires:

- R and S must have the **same number of attributes (same degree)**.
- The attributes must be in the **same order** and each pair of corresponding attributes must be drawn from the **same domain (data type/value range)**.

Example 1 — Checking Attribute Order

- R(Student_ID, Name) and S(Student_ID, Name) → **union-compatible.** Position 1 in both is Student_ID (text-based ID); position 2 in both is Name.
- R(Student_ID, Name) and S(Name, Student_ID) → **NOT union-compatible.** Position 1 in R is an ID, but position 1 in S is a Name — the order doesn't line up, even though both relations have 2 attributes of the same overall types.

> ⬤ **EXAM POINT**
>
> Union compatibility does **not** require the attribute *names* to match — only the **number of attributes and their domain (type)** must correspond.

Example 2 — Attribute Names Don't Need to Match

- R(Student_ID, Name) and S(Roll_No, Full_Name) → **still union-compatible.** Both have 2 attributes; position 1 in each (Student_ID / Roll_No) holds the same kind of ID value, and position 2 in each (Name / Full_Name) holds a person's name.
- Only the column *labels* differ — since the number and domains line up positionally, R ∪ S is valid.

Example 1

Requirement: List every student who is a member of the Sports club, the Music club, or both.

Relation **R = SPORTS_CLUB**(Student_ID, Name)

Student_ID	Name
S1	Aarav
S2	Diya
S3	Kabir

Relation **S = MUSIC_CLUB**(Student_ID, Name)

Student_ID	Name
S2	Diya
S4	Meera
S5	Rohan

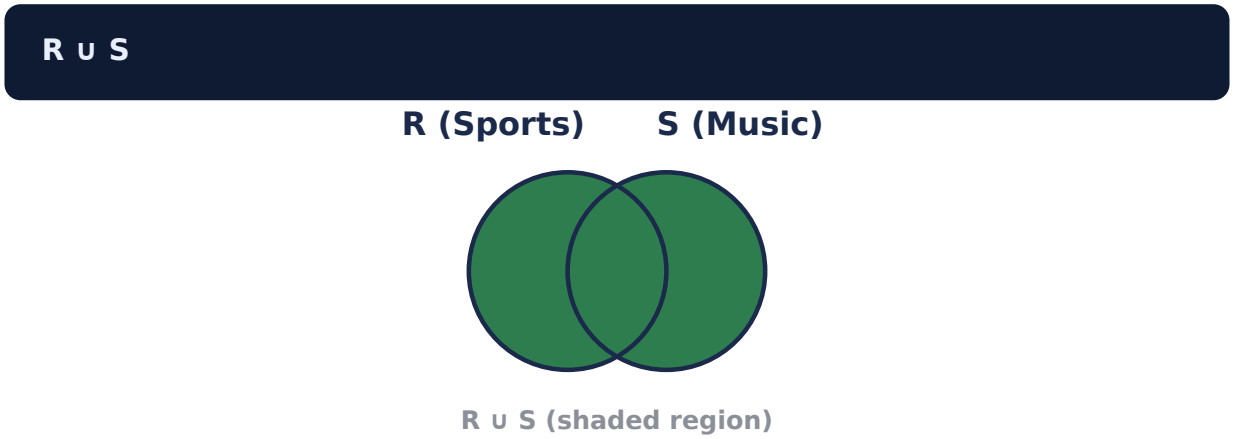


Fig 5.3(a): Union — the entire shaded area is R ∪ S (every tuple from either relation, duplicates removed).

STEP 1. Take every tuple of R: S1-Aarav, S2-Diya, S3-Kabir.

STEP 2. Add every tuple of S: S2-Diya, S4-Meera, S5-Rohan.

STEP 3. S2-Diya occurs in both — keep only **one** copy in the final result.

Result: R ∪ S (5 tuples — duplicate removed)

Student_ID	Name
S1	Aarav
S2	Diya
S3	Kabir
S4	Meera
S5	Rohan

5.4 Intersection (∩)

Meaning

The **Intersection** of two relations R and S, written **R ∩ S**, returns only those tuples that appear in **both** R and S. Like Union, Intersection requires R and S to be **union-compatible**.

General Syntax

```
R ∩ S
```

Example (using the same R and S as Section 5.3)

Requirement: Find students who are members of **both** the Sports club and the Music club.

```
R ∩ S
```

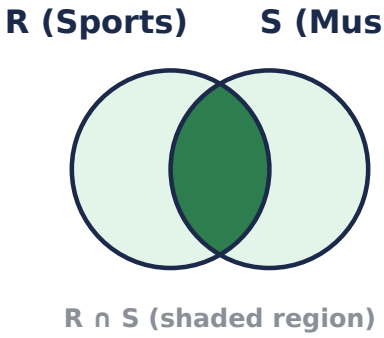


Fig 5.4(a): Intersection — only the darker overlap region is R ∩ S (tuples common to both relations).

STEP 1. Compare every tuple of R against every tuple of S.

STEP 2. S2-Diya is the only tuple present in **both** relations.

Result: R ∩ S

Student_ID	Name
S2	Diya

REMEMBER

R ∩ S can also be computed as **R − (R − S)**, showing that Intersection is expressible using Difference — but for exam purposes, use the ∩ operator directly.

5.5 Difference (−)

Meaning

The **Difference** of two relations, written **R − S**, returns tuples that are present in R but **not** present in S. Difference also requires R and S to be union-compatible.

General Syntax

R − S and S − R

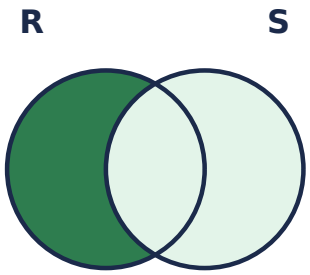
IMPORTANT

Order matters! R − S is **not** the same as S − R. Difference is **not commutative**.

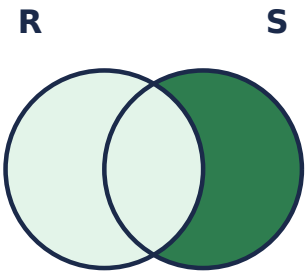
Example (using the same R and S as Section 5.3)

Requirement (a): Students who are in the Sports club but **not** in the Music club → R − S.

Requirement (b): Students who are in the Music club but **not** in the Sports club → S − R.



R − S (shaded region)



S − R (shaded region)

Fig 5.5(a)/(b): R − S (left) shades only R's exclusive region; S − R (right) shades only S's exclusive region — the two results are different.

R − S

Result: R − S

Student_ID	Name
S1	Aarav
S3	Kabir

S − R

Result: S − R

Student_ID	Name
S4	Meera
S5	Rohan

COMMON MISTAKE

A very common exam mistake is writing S − R when the question asks for R − S (or vice-versa). Always identify carefully **which relation the tuples must belong to** and **which relation they must be absent from**.

5.6 Cartesian Product (×)

Meaning

The **Cartesian Product** of two relations R and S, written **R × S**, combines **every tuple of R with every tuple of S**. Unlike Union/Intersection/Difference, R and S do **not** need to be union-compatible — they can have completely different attributes.

General Syntax

```
R × S
```

How Tuples Are Combined

Each resulting tuple is formed by **concatenating** one tuple of R with one tuple of S. If R has attributes (A1, A2, ...) and S has attributes (B1, B2, ...), each output tuple has the combined attribute list (A1, A2, ..., B1, B2, ...).

EXAM POINT

Tuple-count rule: If R has **m** tuples and S has **n** tuples, then R × S has exactly **m × n** tuples, and the degree (number of attributes) of R × S is the **sum** of the degrees of R and S.

Example

Relation **A**(Roll_No, Name)

Roll_No	Name
1	Amit
2	Priya

Relation **B**(Subject)

Subject
Maths
Physics
Chemistry

```
A × B
```

A\B	Maths	Physics	Chemistry
(1,Amit)	((1,Amit),Maths)	((1,Amit),Physics)	((1,Amit),Chemistry)
(2,Priya)	((2,Priya),Maths)	((2,Priya),Physics)	((2,Priya),Chemistry)

2 × 3 = 6 tuples in R × S

Fig 5.6(a): Every tuple of A is paired with every tuple of B → 2 × 3 = 6 combinations.

Result: A × B (2 × 3 = 6 tuples)

Roll_No	Name	Subject
1	Amit	Maths
1	Amit	Physics
1	Amit	Chemistry
2	Priya	Maths
2	Priya	Physics
2	Priya	Chemistry

COMMON MISTAKE

Cartesian Product on its own usually produces many **meaningless combinations** (e.g. pairing a student with an unrelated subject). It becomes meaningful for real queries only when followed by a **Selection** that keeps just the matching combinations — this is exactly the idea used in Simple Join (Section 5.7).

5.7 Simple Join (⋈)

Meaning and Purpose

A **Join** combines related tuples from two relations into single tuples, based on a **join condition** that typically compares a common attribute in both relations (most often, a matching ID). Its purpose is to bring together data that is spread across two relations but logically belongs together.

Relationship Between Cartesian Product and Join

A Join can be understood conceptually as a **Cartesian Product followed by a Selection** that keeps only the combinations satisfying the join condition:

$$R \bowtie_{<condition>} S \quad \text{is equivalent to} \quad \sigma_{<condition>}(R \times S)$$

General Syntax (Simple Join)

$$R \bowtie_{<join\ condition>} S$$

The join condition usually has the form **R.attribute θ S.attribute**, where θ is a comparison operator (most commonly =).

IMPORTANT

As per the syllabus, only **Simple Join** is covered here. Natural Join, Equi Join, Theta Join and Outer Join are **not part of this topic** and are not discussed.

Example

Relation **STUDENT**(Student_ID, Name, Dept_ID)

Student_ID	Name	Dept_ID
S1	Aarav	D1
S2	Diya	D2
S3	Kabir	D1
S4	Meera	D3

Relation **DEPARTMENT**(Dept_ID, Dept_Name)

Dept_ID	Dept_Name
D1	CSE
D2	ECE
D3	ME

Requirement: Combine each student's record with the name of their department.

Join Condition

STUDENT.Dept_ID = DEPARTMENT.Dept_ID

Relational Algebra Expression

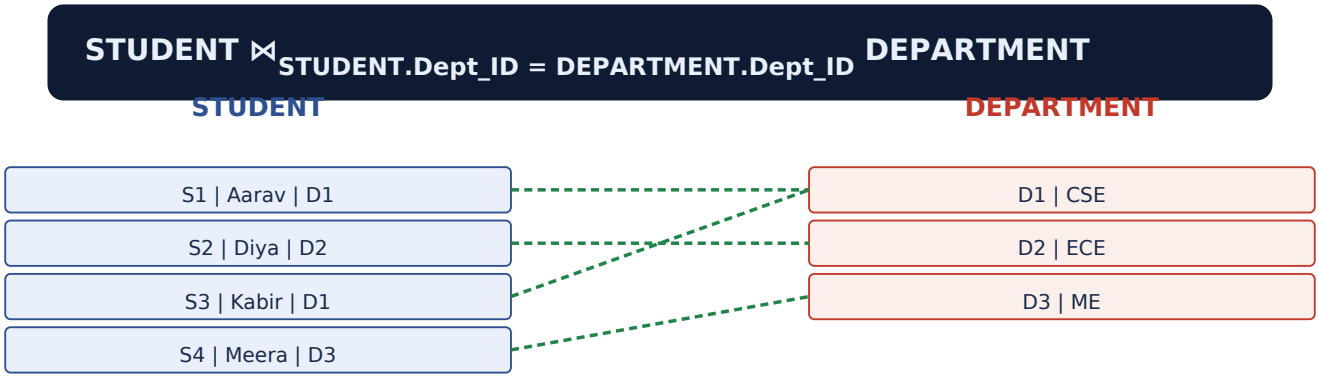


Fig 5.7(a): Dashed lines connect STUDENT tuples to the DEPARTMENT tuple whose Dept_ID matches — these matched pairs are combined in the result.

STEP 1. Form the Cartesian Product STUDENT × DEPARTMENT (conceptually).

STEP 2. Apply the condition STUDENT.Dept_ID = DEPARTMENT.Dept_ID, keeping only matching combinations.

STEP 3. Each surviving combination becomes one tuple of the result, containing attributes from both relations.

Result: STUDENT ⋈ DEPARTMENT

Student_ID	Name	Dept_ID	Dept_Name
S1	Aarav	D1	CSE
S2	Diya	D2	ECE
S3	Kabir	D1	CSE
S4	Meera	D3	ME

● EXAM POINT

A Simple Join lets us combine **related** information from two relations in one step, instead of generating the full Cartesian Product and filtering it separately.

5.8 Queries to be Solved Based on the Above Operations

This section uses one small, consistent database schema for every practice query. All solutions use **only** Selection, Projection, Union, Intersection, Difference, Cartesian Product and Simple Join — no SQL is used anywhere.

Schema Used in this Section

STUDENT(Student_ID, Name, Dept_ID, Marks)

DEPARTMENT(Dept_ID, Dept_Name)

COURSE(Course_ID, Course_Name, Dept_ID)

STUDENT

Student_ID	Name	Dept_ID	Marks
S1	Aarav	D1	85
S2	Diya	D2	72
S3	Kabir	D1	91
S4	Meera	D3	68
S5	Rohan	D2	78
S6	Sara	D1	55

DEPARTMENT

Dept_ID	Dept_Name
D1	CSE
D2	ECE
D3	ME

COURSE

Course_ID	Course_Name	Dept_ID
C1	Data Structures	D1
C2	Digital Circuits	D2
C3	Thermodynamics	D3
C4	Algorithms	D1

QUERY 1

Question: Find the details of all students who scored more than 80 marks.

Given Relations: STUDENT

Relational Algebra Expression:

$$\sigma_{Marks>80}(STUDENT)$$

Step-by-Step Solution:

STEP 1. Scan every tuple of STUDENT.

STEP 2. Keep tuples where Marks > 80: this matches S1, S3.

STEP 3. Return the matching tuples with all attributes.

Final Result

Student_ID	Name	Dept_ID	Marks
S1	Aarav	D1	85
S3	Kabir	D1	91

QUERY 2

Question: List the Name and Marks of every student belonging to department D1.

Given Relations: STUDENT

Relational Algebra Expression:

$$\pi_{Name,\ Marks}(\sigma_{Dept_ID='D1'}(STUDENT))$$

Step-by-Step Solution:

STEP 1. Inner step: σ Dept_ID='D1' (STUDENT) selects S1, S3, S6.

STEP 2. Outer step: π Name, Marks(...) keeps only these two columns.

STEP 3. Return the resulting two-column relation.

Final Result

Name	Marks
Aarav	85
Kabir	91
Sara	55

QUERY 3

Question: List (Student_ID, Name) of students who either scored more than 80 marks, or belong to department D1, or both.

Given Relations: Two intermediate union-compatible relations derived from STUDENT

Relational Algebra Expression:

$$\pi_{Student_ID,Name}(\sigma_{Marks>80}(STUDENT)) \cup \pi_{Student_ID,Name}(\sigma_{Dept_ID='D1'}(STUDENT))$$

Step-by-Step Solution:

STEP 1. Compute R = π Student_ID,Name (σ Marks>80 (STUDENT)) = {S1-Aarav, S3-Kabir}.

STEP 2. Compute S = π Student_ID,Name (σ Dept_ID='D1' (STUDENT)) = {S1-Aarav, S3-Kabir, S6-Sara}.

STEP 3. R and S are union-compatible (same attributes) — compute R ∪ S.

STEP 4. Duplicate tuple S1-Aarav / S3-Kabir appear in both — keep a single copy each.

Final Result

Student_ID	Name
S1	Aarav
S3	Kabir
S6	Sara

QUERY 4

Question: List (Student_ID, Name) of students who scored more than 80 marks AND belong to department D1.

Given Relations: Same intermediate relations R and S as Query 3

Relational Algebra Expression:

$$\pi_{Student_ID,Name}(\sigma_{Marks>80}(STUDENT)) \cap \pi_{Student_ID,Name}(\sigma_{Dept_ID='D1'}(STUDENT))$$

Step-by-Step Solution:

STEP 1. R = {S1-Aarav, S3-Kabir} (Marks > 80).

STEP 2. S = {S1-Aarav, S3-Kabir, S6-Sara} (Dept_ID = 'D1').

STEP 3. R ∩ S keeps only tuples common to both sets.

Final Result

Student_ID	Name
S1	Aarav
S3	Kabir

QUERY 5

Question: List (Student_ID, Name) of students who belong to department D1 but did NOT score more than 80 marks.

Given Relations: Same intermediate relations R and S as Query 3

Relational Algebra Expression:

$$\pi_{Student_ID,Name}(\sigma_{Dept_ID='D1'}(STUDENT)) - \pi_{Student_ID,Name}(\sigma_{Marks>80}(STUDENT))$$

Step-by-Step Solution:

STEP 1. S (Dept_ID='D1') = {S1-Aarav, S3-Kabir, S6-Sara}.

STEP 2. R (Marks>80) = {S1-Aarav, S3-Kabir}.

STEP 3. S − R removes from S every tuple that also occurs in R, leaving only S6-Sara.

STEP 4. Note the order: this is S − R, not R − S.

Final Result

Student_ID	Name
S6	Sara

QUERY 6

Question: Generate all possible (Department, Course) combinations, regardless of whether the course actually belongs to that department.

Given Relations: DEPARTMENT, COURSE

Relational Algebra Expression:

$$DEPARTMENT \times COURSE$$

Step-by-Step Solution:

STEP 1. DEPARTMENT has 3 tuples, COURSE has 4 tuples.

STEP 2. By the tuple-count rule, DEPARTMENT × COURSE has 3 × 4 = 12 tuples.

STEP 3. Every DEPARTMENT tuple is concatenated with every COURSE tuple — most combinations are not meaningful (e.g. pairing D1-CSE with C2-Digital Circuits, which actually belongs to D2).

Final Result

Dept_ID	Dept_Name	Course_ID	Course_Name	Dept_ID
D1	CSE	C1	Data Structures	D1
D1	CSE	C2	Digital Circuits	D2
D1	CSE	C3	Thermodynamics	D3
D1	CSE	C4	Algorithms	D1
D2	ECE	C1	Data Structures	D1
D2	ECE	C2	Digital Circuits	D2
D2	ECE	C3	Thermodynamics	D3
D2	ECE	C4	Algorithms	D1
D3	ME	C1	Data Structures	D1

D3	ME	C2	Digital Circuits	D2
D3	ME	C3	Thermodynamics	D3
D3	ME	C4	Algorithms	D1

QUERY 7

Question: List the Student_ID, Name and Dept_Name for every student (i.e. resolve each student's department name).

Given Relations: STUDENT, DEPARTMENT

Relational Algebra Expression:

$$\text{STUDENT} \bowtie_{\text{STUDENT.Dept_ID = DEPARTMENT.Dept_ID}} \text{DEPARTMENT}$$

Step-by-Step Solution:

STEP 1. Conceptually form STUDENT × DEPARTMENT.

STEP 2. Keep only combinations where STUDENT.Dept_ID = DEPARTMENT.Dept_ID.

STEP 3. Each STUDENT tuple is now combined with the single DEPARTMENT tuple that matches it.

Final Result

Student_ID	Name	Dept_ID	Marks	Dept_Name
S1	Aarav	D1	85	CSE
S2	Diya	D2	72	ECE
S3	Kabir	D1	91	CSE
S4	Meera	D3	68	ME
S5	Rohan	D2	78	ECE
S6	Sara	D1	55	CSE

QUERY 8

Question: List the names of students who are in the CSE department and scored more than 80 marks (using Join, Selection and Projection together).

Given Relations: STUDENT, DEPARTMENT

Relational Algebra Expression:

$$\pi_{\text{Name}}(\sigma_{\text{Dept_Name='CSE'} \wedge \text{Marks}>80}(\text{STUDENT} \bowtie_{\text{STUDENT.Dept_ID=DEPARTMENT.Dept_ID}} \text{DEPARTMENT}))$$

Step-by-Step Solution:

STEP 1. Innermost: join STUDENT with DEPARTMENT on Dept_ID, giving each student their Dept_Name (as in Query 7).

STEP 2. Middle: apply σ Dept_Name='CSE' ∧ Marks>80 on the joined relation — keeps only S1-Aarav (85, CSE) and S3-Kabir (91, CSE).

STEP 3. Outer: apply π Name to keep only the Name attribute.

Final Result

Name
Aarav
Kabir

QUERY 9

Question: List the names of all courses offered by department D1 or department D2.

Given Relations: COURSE

Relational Algebra Expression:

$$\pi_{\text{Course_Name}}(\sigma_{\text{Dept_ID='D1'}}(\text{COURSE})) \cup \pi_{\text{Course_Name}}(\sigma_{\text{Dept_ID='D2'}}(\text{COURSE}))$$

Step-by-Step Solution:

STEP 1. Compute courses of D1: σ Dept_ID='D1' (COURSE), then project Course_Name — {Data Structures, Algorithms}.

STEP 2. Compute courses of D2: σ Dept_ID='D2' (COURSE), then project Course_Name — {Digital Circuits}.

STEP 3. Both intermediate relations have the single attribute Course_Name, so they are union-compatible — take their Union.

Final Result

Course_Name
Data Structures
Algorithms
Digital Circuits

5.9 Quick Revision Tables

Operation Summary

Operation	Symbol	Works Mainly On	Purpose	Output
Selection	σ	Rows (tuples)	Filter tuples satisfying a condition	Same attributes, fewer/equal rows
Projection	π	Columns (attributes)	Choose specific attributes	Fewer/equal columns, duplicates removed
Union	∪	Two union-compatible relations	All tuples from R or S	Combined tuples, duplicates removed
Intersection	∩	Two union-compatible relations	Common tuples of R and S	Only tuples present in both
Difference	−	Two union-compatible relations	Tuples in R but not in S	Order-sensitive result
Cartesian Product	×	Any two relations	Pair every tuple of R with every tuple of S	m × n tuples; attributes = sum of both
Simple Join	⋈	Two relations with a join condition	Combine related tuples	Filtered, meaningful combinations

Union vs Intersection vs Difference

Aspect	Union (R∪S)	Intersection (R∩S)	Difference (R−S)
Result contains	All tuples of R and S	Only common tuples	Tuples only in R
Commutative?	Yes: R∪S = S∪R	Yes: R∩S = S∩R	No: R−S ≠ S−R
Compatibility needed?	Yes	Yes	Yes
Duplicate handling	Removed	N/A (only common kept)	N/A

5.10 Final Exam Revision Notes

The definitions, rules and common mistakes for each operation have already been explained in Sections 5.1–5.7 (and summarised in the tables of Section 5.9). Rather than repeating them, this final section gives a **step-by-step method for solving any query** and a **pre-submission checklist** to run through in the exam hall.

Step-by-Step Approach for Solving Any Query

- Read the requirement carefully and identify **which relation(s)** are involved.
- Decide whether rows need to be filtered (σ), columns need to be chosen (π), or two relations need to be combined (∪, ∩, −, ×, or Join).
- For queries needing data from **two different relations** that are related by a common attribute, use a **Simple Join**.
- Build the expression from the **innermost** operation outward, and verify the schema (attribute list) of each intermediate result.
- Double-check union compatibility whenever using ∪, ∩, or −.
- Write out the final result relation to confirm it answers the original question.

Pre-Submission Checklist

- Have I used **only** the seven operations covered in this chapter (σ, π, ∪, ∩, −, ×, Join) — no SQL, Division, or other operators?
- Is every relation I combined with ∪, ∩ or − actually union-compatible?
- For a Difference query, have I written the operands in the **order the question asks for**?
- If my expression ends in a Projection, have duplicate tuples been accounted for?
- Does the **degree and tuple count** of my final relation make sense given the operations applied (e.g. m × n for a Cartesian Product, before any filtering)?
- Have I labelled the final answer as a clear relation/table, not just a bare expression?

● EXAM POINT

The fastest way to build confidence for this topic is to re-derive Queries 1–9 of Section 5.8 from scratch, without looking at the given solutions, and only then check your working against the steps shown there.